

DATA 266 Homework 2: Embedding Transfer Learning, Retrieval-Augmented Generation, and Training Optimization

Haritha Gottumukkala
Department of Applied Data Science
San Jose State University
San Jose, California, USA

DATA 266 – Generative AI and Large Language Models
SID4: 1598 SEED: 1598

Abstract

In this homework, I worked with three areas that connect directly to modern generative AI systems: adapting pretrained word embeddings to a new domain, building a Retrieval-Augmented Generation (RAG) pipeline, and comparing training optimization techniques on a GPU. For Part 1, I started with the pretrained word2vec-google-news-300 model, transferred the available vectors into a trainable Word2Vec vocabulary built from 25,000 IMDB reviews, and fine-tuned the embeddings for five epochs. I compared the top-3 nearest neighbors of cast, score, plot, screen, and review before and after fine-tuning. The original-to-fine-tuned cosine similarities showed that review shifted the most (0.5391), while screen shifted the least (0.6706). For Part 2, I loaded ten Wikipedia movie pages with LangChain, split them using the required 500/50 chunking configuration, embedded 3,573 chunks with MiniLM, stored them in FAISS, and used an explicit retriever-prompt-LLM pipeline with FLAN-T5-Large. All five questions had answer-bearing evidence within the top three retrieved chunks, giving a 100% retrieval success rate, although the Titanic and Matrix questions placed the direct answer at Rank 2. For Part 3, I compared CPU-to-GPU data transfer, GPU-resident data, Kaiming initialization, activation checkpointing, gradient accumulation, and mixed precision under the same controlled model and training settings. The results showed that the benefit of an optimization technique depends on workload size: some techniques added overhead rather than improving time or measured memory in this small experiment.

Index Terms— Word2Vec, transfer learning, IMDB, Retrieval-Augmented Generation, RAG, FAISS, sentence embeddings, FLAN-T5, activation checkpointing, gradient accumulation, mixed precision.

I. INTRODUCTION

This homework brought together three topics that I had previously studied mostly as separate ideas. The first was transfer learning for word embeddings. Word2Vec learns continuous vector representations from surrounding context, so words that are used in similar contexts tend to move closer together in the embedding space [1]. Instead of learning every vector from scratch, I used a pretrained Google News model as the starting point and then continued training on IMDB movie-review text. This made it possible to observe how general-purpose word relationships changed after exposure to a more specific domain.

The second topic was Retrieval-Augmented Generation. A

RAG system first retrieves relevant external text and then gives that evidence to a language model as context for generation [3]. In this experiment, the external knowledge source was a set of ten Wikipedia movie pages. I implemented the pipeline explicitly so that document loading, chunking, embedding, vector search, prompting, and answer generation could each be inspected separately.

The final topic was training optimization. I compared five techniques using the same neural-network architecture, synthetic data, effective batch size, optimizer, learning rate, random seed, and number of training steps. My goal was not to assume that every technique would improve performance, but to measure how it behaved in the controlled setup used in the notebook.

Throughout the homework, I checked intermediate outputs instead of relying only on whether a cell executed successfully. This included confirming vector transfer, manually reading retrieved passages, verifying the rank of the first answer-bearing chunk, sanity-checking the generation model, and repeating GPU benchmarks after a warm-up run.

II. EXPERIMENTAL SETUP AND REPRODUCIBILITY

The personal parameters and execution environment used in the final notebook are summarized in Table I.

TABLE I. Personal Parameters and Runtime

Item	Value
SID4 / SEED	1598 / 1598
SLICE / HP_ID	598 / 2
CLS_A / CLS_B	8 / 5
Runtime	Google Colab
GPU	NVIDIA Tesla T4
Python	3.13.15
PyTorch	2.11.0+cu128
Gensim	4.4.0
Final run time	2026-09-06 17:40:34

SEED = 1598 was used for Python, NumPy, PyTorch, and CUDA where randomness was involved. Word2Vec was trained with one worker so that the fine-tuning run would be more reproducible. The final notebook output, rather than earlier development runs, is used for every numeric result in this report.

III. EMBEDDING-BASED TRANSFER LEARNING

A. Pretrained Model and IMDB Preparation

I loaded word2vec-google-news-300, which represents words using 300-dimensional vectors. The required target

words were *cast*, *score*, *plot*, *screen*, and *review*. All five were present in the pretrained vocabulary. Before fine-tuning, I saved their original vectors and extracted each word's top three nearest neighbors and cosine similarity scores.

For the adaptation corpus, I used the 25,000-review IMDB training split [2]. HTML markers were removed and the text was converted into lowercase tokens with Gensim preprocessing. The trainable IMDB vocabulary contained 46,149 words. Of these, 36,450 were also present in the Google News vocabulary, giving 78.98% pretrained-vector coverage. The pretrained vectors were copied into the trainable Word2Vec model for all shared words, and I explicitly confirmed successful transfer for all five target words.

The target-word counts were 3,829 for *cast*, 1,030 for *score*, 6,589 for *plot*, 2,492 for *screen*, and 850 for *review*. I then fine-tuned the transferred embeddings on the IMDB training reviews for five epochs.

B. Nearest Neighbors Before and After Fine-Tuning

Table II contains the full before-and-after comparison required by the assignment.

The clearest changes were visible for *cast* and *plot*. Before fine-tuning, their neighbors were dominated by grammatical variations such as *casts*, *casting*, *plots*, and *plotting*. After training on movie reviews, *cast* became close to *supporting*, *actors*, and *ensemble*, while *plot* became close to *storyline* and *story*. These changes were consistent with the movie-review domain.

I also observed that not every new neighbor was clearly meaningful; for example, *piedras* appeared near *screen* after fine-tuning.

C. Original-to-Fine-Tuned Vector Shift

For each target word, I computed cosine similarity between its saved original vector $\mathbf{v}_o$ and fine-tuned vector $\mathbf{v}_f$:

$$\cos(\mathbf{v}_o, \mathbf{v}_f) = \frac{\mathbf{v}_o \cdot \mathbf{v}_f}{\|\mathbf{v}_o\| \|\mathbf{v}_f\|}. \quad (1)$$

TABLE III. Original-to-Fine-Tuned Vector Similarity

Word	Cosine Similarity
cast	0.6646
score	0.5624
plot	0.6457
screen	0.6706
review	0.5391

A lower value indicates a larger directional change. Therefore, *review* shifted the most at 0.5391, while *screen* shifted the least at 0.6706.

One result I found interesting was that *review* had the lowest frequency of the five target words but still moved the most. This showed that raw frequency alone was not enough to explain the amount of embedding change.

D. t-SNE Visualization

I projected the original and fine-tuned vectors into two dimensions using t-SNE [5]. The notebook used perplexity 3, PCA initialization, and random state 1598.

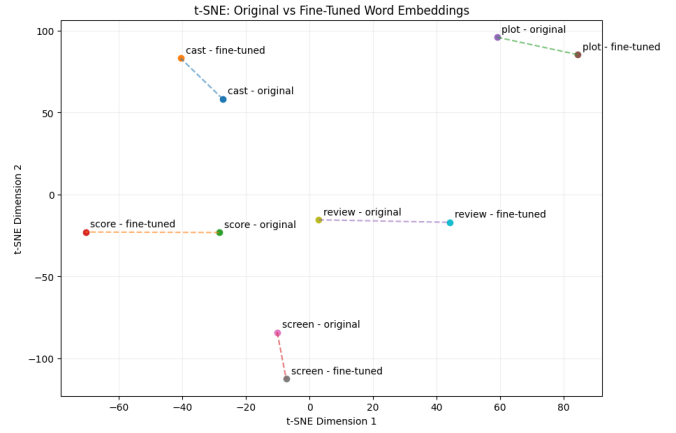


Fig. 1. t-SNE visualization of the original and fine-tuned embeddings for the five target words.

Fig. 1 provides a visual view of the shifts, but I used the cosine values in Table III for the actual most/least-shifted comparison because t-SNE changes the geometry while reducing 300 dimensions to two.

IV. RETRIEVAL-AUGMENTED GENERATION

A. Document Loading, Chunking, and Embeddings

I used ten Wikipedia movie pages: *Inception*, *Titanic* (1997 film), *The Dark Knight*, *Interstellar*, *The Matrix*, *Avatar* (2009 film), *The Godfather*, *Jurassic Park*, *Toy Story*, and *Gladiator* (2000 film).

The pages were loaded with LangChain's `WebBaseLoader`; all ten loaded successfully.

The required main splitter used a chunk size of 500 and overlap of 50. It produced 3,573 chunks. Individual chunk counts ranged from 189 for *Gladiator* to 537 for *Jurassic Park*.

I embedded the chunks using `sentence-transformers/all-MiniLM-L6-v2`. Each query embedding had 384 dimensions. All 3,573 chunk embeddings were stored in FAISS [4], and the retriever returned the top three chunks for each question.

TABLE IV. Main RAG Configuration

Component	Setting
Documents	10 Wikipedia movie pages
Chunk size / overlap	500 / 50
Total chunks	3573
Embedding model	all-MiniLM-L6-v2
Embedding dimension	384
Vector store	FAISS
Retriever	top-k = 3
Generator	google/flan-t5-large

B. Explicit RAG Pipeline

The assignment required explicit components rather than a single pre-built chain. I therefore implemented the sequence as

Question $\rightarrow$ Retriever $\rightarrow$ Prompt $\rightarrow$ LLM $\rightarrow$ Answer. (2)

The retriever obtained the top three FAISS chunks. Their text was joined into the prompt context, and the prompt told the model to answer only from that context.

TABLE II. Top-3 Nearest Neighbors Before and After IMDB Fine-Tuning

Word	Rank	Neighbor Before	Cosine	Neighbor After	Cosine
cast	1	casts	0.7219	supporting	0.6074
cast	2	casting	0.7188	actors	0.5672
cast	3	Cast	0.6638	ensemble	0.5554
score	1	scoring	0.7197	nicolai	0.6278
score	2	scores	0.6596	music	0.6146
score	3	scored	0.6384	ennio	0.5992
plot	1	plots	0.7625	storyline	0.6451
plot	2	Plot	0.6524	plots	0.6260
plot	3	plotting	0.6328	story	0.6125
screen	1	screens	0.7729	screens	0.5509
screen	2	onscreen	0.6115	onscreen	0.4936
screen	3	LCD_screen	0.5599	piedras	0.4413
review	1	reviewed	0.6630	comment	0.6392
review	2	reviewing	0.6610	reviews	0.5871
review	3	reviews	0.6380	comments	0.5545

FLAN-T5-Large was loaded directly with `AutoTokenizer` and `AutoModelForSeq2SeqLM`.

Before the final evaluation, I asked the model for the capital of France and verified that it returned *Paris*.

C. Retrieved Evidence and Generated Answers

The notebook prints the complete text of all three retrieved chunks for every question. To keep the report readable while still displaying the retrieval result, Table V gives a concise description of each retrieved chunk together with its source and whether it contains the direct answer.

All five questions had at least one answer-bearing passage inside the top three retrieved chunks. Therefore,

$$\text{Retrieval Success Rate} = \frac{5}{5} \times 100 = 100\%. \quad (3)$$

The result was strong at top-3, but it was not perfect at Rank 1. The Titanic and Matrix questions are analyzed as retrieval-ranking failures in Section 6.

D. Alternate Chunking Comparison

For the Titanic and Matrix questions, I changed the splitter to chunk size 300 and overlap 75. The alternate splitter produced 6,416 chunks.

Table VII compares the original and alternate retrievals.

The smaller chunks changed the surrounding content but did not move the direct answer above Rank 2 in either case. The generated answers also stayed the same. In this experiment, changing chunk size and overlap alone did not fix the ranking issue.

V. TRAINING OPTIMIZATION EXPERIMENTS

A. Controlled Model and Benchmark Procedure

The optimization experiments used the same `ExperimentNetwork`, synthetic dataset, effective batch size, training steps, Adam optimizer, learning rate, and SEED.

The model used input dimension 512, three 1024-unit hidden layers with ReLU activations, and 10 output classes. The dataset contained 4,096 samples.

A batch of 64 produced an output tensor of shape [64, 10], confirming the model dimensions.

TABLE VIII. Controlled Training Settings

Setting	Value
Model	ExperimentNetwork
Input / hidden dims.	512 / 1024
Classes	10
Dataset size	4096
Effective batch size	64
Training steps	50
Optimizer	Adam
Learning rate	0.001
Seed	1598
GPU	Tesla T4
Benchmark	1 warm-up + 3 measured runs

For each measured run, CUDA was synchronized around the timed section and peak allocated GPU memory was recorded. Final loss was also retained as the training-performance measure.

B. Representative Code for the Five Techniques

The full implementation is in the submitted notebook. The following short snippets show the specific change made for each technique.

Listing 1: Basic implementation snippets used in Part 3.

```
# 1) Tensor placement
X_source = X_data.to(DEVICE)
y_source = y_data.to(DEVICE)

# 2) Kaiming initialization
nn.init.kaiming_normal_(
    layer.weight,
    nonlinearity="relu"
)

# 3) Activation checkpointing
x = checkpoint(
    self.hidden_block_1,
    x,
    use_reentrant=False
)

# 4) Gradient accumulation
loss_for_backward = (
    loss / accumulation_steps
)

scaler.scale(
    loss_for_backward
).backward()

scaler.step(optimizer)
scaler.update()

# 5) Mixed precision
with torch.amp.autocast(
```

TABLE V. Top-3 Retrieved Chunks for the Five Evaluation Questions

Q	Rank	Source	Chunk	Retrieved Evidence / Content	Direct Answer?
1	1	Inception	10	Film is “written and directed by Christopher Nolan.”	Yes
1	2	Inception	147	Review commentary; does not identify the director.	No
1	3	Inception	198	Reference section mentioning an article about the film’s director.	No
2	1	Titanic	336	Discusses a real grave labeled “J. Dawson” in Halifax.	No
2	2	Titanic	335	“Leonardo DiCaprio as Jack Dawson” followed by character details.	Yes
2	3	Titanic	342	Describes Spicer Lovejoy and Jack’s rescue of Rose.	No
3	1	The Dark Knight	1121	Citation title explicitly says “Heath Ledger’s Joker.”	Yes
3	2	The Dark Knight	858	Casting section discussing Rachel Dawes.	No
3	3	The Dark Knight	1093	Joker-related citations without the direct actor statement.	No
4	1	The Matrix	1550	States only that Morpheus offers Neo a choice between two pills.	No
4	2	The Matrix	1551	States that Neo takes the red pill and awakens in the real world.	Yes
4	3	The Matrix	1851	Topic list containing “Red pill and blue pill.”	No
5	1	Gladiator	3437	Score was composed by Hans Zimmer and Lisa Gerrard.	Yes
5	2	Gladiator	3517	Wikiquote reference only.	No
5	3	Gladiator	3492	Behind-the-scenes and script references unrelated to the answer.	No

TABLE VI. Manual Retrieval Evaluation and Final Generated Answers

Question	Top-3 Success	First Relevant Rank	Generated Answer
Who directed Inception?	Yes	1	Christopher Nolan
Who played Jack Dawson in Titanic?	Yes	2	Leonardo DiCaprio
Which actor played the Joker in The Dark Knight?	Yes	1	Heath Ledger
What color pill does Neo take in The Matrix?	Yes	2	red
Who composed the music for Gladiator?	Yes	1	Hans Zimmer and Lisa Gerrard

```

device_type=DEVICE.type,
dtype=torch.float16,
enabled=use_amp
):
    outputs = model(xb)

```

C. Measured Results

Table IX shows the pairwise controlled measurements produced by the final notebook. This table is useful because each technique can be compared directly with its corresponding control run.

The consolidated notebook summary is shown in Table X.

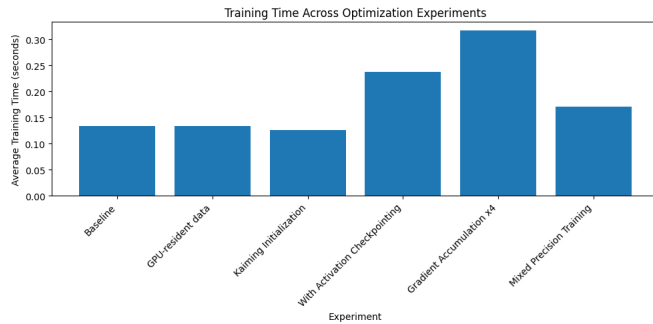


Fig. 2. Average training time for the baseline and optimization configurations.

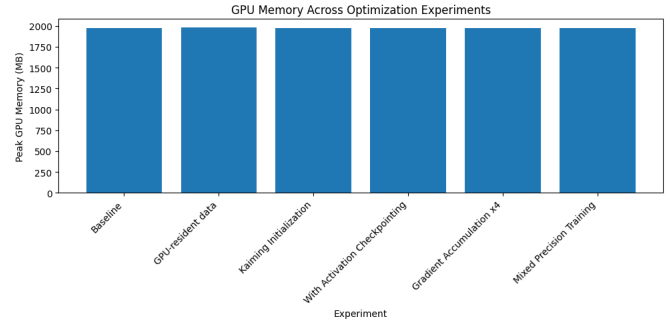


Fig. 3. Average peak GPU memory measured for the baseline and optimization configurations.

D. Findings from the Controlled Comparisons

Tensor placement: The CPU-to-GPU batch run averaged 0.125763 s, while the GPU-resident version averaged 0.133244 s. The final loss was identical, but the GPU-resident run used more measured memory. For this small dataset, keeping the full dataset on the GPU did not improve measured runtime.

Weight initialization: Kaiming initialization averaged 0.125693 s compared with 0.128300 s for the default-initialization control. Its final loss was higher (2.326585 versus 2.303293). Because the benchmark used only 50 training steps, I interpret the loss difference as a change in early optimization behavior rather than as a final model-quality conclusion.

Activation checkpointing: Runtime increased from 0.127737 s without checkpointing to 0.237984 s with checkpointing, while measured peak memory did not decrease. The extra time is consistent with recomputing activations during backpropagation; the model was small enough that a memory advantage was not visible in this run [6].

Gradient accumulation: Accumulation ×4 kept the effective batch size at 64 by processing four micro-batches of 16. Runtime increased from 0.138447 s to 0.317288 s, while final loss stayed the same and measured peak memory was almost unchanged. Since a batch of 64 already fit on the GPU, the main effect here

TABLE VII. Original 500/50 and Alternate 300/75 Chunking Comparison

Question	Config.	First Relevant Rank	Top-3 Retrieval Pattern	Answer
Jack Dawson / Titanic	500/50	2	J. Dawson grave → Leonardo DiCaprio → Spicer Lovejoy	Leonardo DiCaprio
Jack Dawson / Titanic	300/75	2	J. Dawson grave → Leonardo DiCaprio → Jack Dawson-related survivor text	Leonardo DiCaprio
Neo pill / The Matrix	500/50	2	two-pill choice → Neo takes red pill → Matrix topic list	red
Neo pill / The Matrix	300/75	2	two-pill choice → Neo takes red pill → red-pill interpretation text	red

TABLE IX. Pairwise Controlled Optimization Measurements

Comparison	Configuration	Avg. Time (s)	Peak GPU Memory (MB)	Final Loss
Tensor placement	CPU-to-GPU batches	0.125763	1975.382324	2.303293
Tensor placement	GPU-resident data	0.133244	1983.413574	2.303293
Initialization	Default initialization	0.128300	1975.382324	2.303293
Initialization	Kaiming initialization	0.125693	1975.382324	2.326585
Checkpointing	Without checkpointing	0.127737	1975.382324	2.303293
Checkpointing	With checkpointing	0.237984	1976.382324	2.303293
Gradient accumulation	No accumulation	0.138447	1976.382324	2.303293
Gradient accumulation	Accumulation ×4	0.317288	1976.380859	2.303293
Precision	FP32 training	0.126003	1976.382324	2.303293
Precision	Mixed precision	0.171087	1976.382812	2.301819

was additional forward/backward-pass overhead.

Mixed precision: FP32 averaged 0.126003 s and mixed precision averaged 0.171087 s. The final losses were very close (2.303293 and 2.301819). Mixed precision therefore preserved the training signal in this short experiment but did not produce a speed advantage for this workload [7].

VI. FAILURE AND ERROR ANALYSIS

The assignment specifically required at least two RAG failures. A separate six-item failure gallery was not required, so I focused on the two ranking problems observed in the five evaluated questions.

A. Failure 1: Titanic Retrieval Ranking

For *Who played Jack Dawson in Titanic?*, Rank 1 came from the correct Titanic page but discussed a real grave marked “J. Dawson” in Halifax. It was semantically related to the query but did not answer the actor question.

Rank 2 contained the direct passage “Leonardo DiCaprio as Jack Dawson.” This is a ranking failure because a less useful related passage was placed above the answer-bearing passage.

The LLM still answered correctly because Rank 2 was included in the top-3 context.

B. Failure 2: Matrix Retrieval Ranking

For *What color pill does Neo take in The Matrix?*, Rank 1 only stated that Morpheus offered Neo a choice between two pills.

The passage containing the exact fact—“Neo takes the red pill”—appeared at Rank 2. This is a partially relevant passage being ranked above the direct answer.

Again, top-3 retrieval prevented the ranking issue from becoming a final generation error.

C. What the Alternate Chunking Showed

Changing from 500/50 to 300/75 did not improve either failure.

Both answer-bearing passages remained at Rank 2, and both final generated answers remained correct. This showed that smaller chunks alone were not enough to solve these two ranking issues.

D. Development Errors and Corrections

I also encountered several implementation problems while developing the notebook.

The original IMDB loader call was changed to

```
load_dataset("stanfordnlp/imdb")
```

after the shorter dataset identifier failed in the current environment.

A Wikipedia loader path produced a JSON parsing error, so I switched to `WebBaseLoader` with exact Wikipedia URLs.

An early `text2text-generation` pipeline configuration also failed, so FLAN-T5 was loaded directly with `AutoTokenizer` and `AutoModelForSeq2SeqLM`.

One of the most useful checks was a model-output failure rather than a Python exception. An earlier FLAN-T5-base setup returned *london* for the capital of France. I changed to FLAN-T5-Large and reran the same sanity check; the corrected setup returned *Paris*.

After a Colab runtime reset, I also encountered missing-variable/import errors such as `optimization_results` and `plt` being undefined.

Rerunning the setup cells in dependency order restored the final experiment state.

VII. DISCUSSION

The three parts produced different kinds of measurements, but they all reinforced the same idea: actual behavior has to be checked in the context where the method is being used.

In Part 1, the before-and-after nearest-neighbor table gave a qualitative view of domain adaptation, while original-to-fine-tuned cosine similarity gave a more direct numerical measure. The shift of *plot* toward *storyline* and *story*, and of *cast* toward *actors* and *ensemble*, made the movie-review domain visible in the learned neighborhoods.

At the same time, unexpected neighbors showed that fine-tuning does not make every local relationship cleaner.

In Part 2, the 100% top-3 retrieval success rate looked perfect at first, but manually checking the ranks revealed two weaknesses. This is why I report both top-3 success and first relevant rank.

The alternate chunking experiment also showed that one parameter change did not automatically improve retrieval quality. A different embedding model, query reformulation, metadata filtering, or a reranking stage would be reasonable next experiments.

In Part 3, the controlled measurements did not support a sim-

TABLE X. Final Optimization Summary from the Executed Notebook

Experiment	Avg. Time (s)	Peak GPU Memory (MB)	Final Loss
Baseline	0.1337	1975.38	2.3033
GPU-resident data	0.1332	1983.41	2.3033
Kaiming initialization	0.1257	1975.38	2.3266
Activation checkpointing	0.2380	1976.38	2.3033
Gradient accumulation $\times 4$	0.3173	1976.38	2.3033
Mixed precision	0.1711	1976.38	2.3018

ple “optimization always makes training faster” interpretation.

Kaiming initialization had the lowest time in the consolidated summary but a higher final loss.

Checkpointing and gradient accumulation added substantial runtime overhead without a clear measured memory benefit.

Mixed precision kept loss close to FP32 but was slower in this short T4 benchmark.

The workload was small, so techniques that are valuable at larger model scales did not necessarily show the same advantage here.

A limitation of the timing experiment is that the measured control runs varied slightly even when their settings were nearly identical. For this reason, I used a warm-up plus three measured runs and avoided treating very small differences as universal conclusions.

The optimization findings should be interpreted as results from this controlled notebook workload, not as general benchmarks for all neural networks.

VIII. CONCLUSION

This homework helped me connect transfer learning, retrieval-augmented generation, and training optimization through experiments that I could inspect directly.

For embedding transfer learning, I successfully transferred pretrained Google News vectors into a trainable IMDB Word2Vec model and fine-tuned them for five epochs. The nearest-neighbor relationships changed after domain adaptation, and the vector comparison showed that *review* shifted the most while *screen* shifted the least.

For RAG, I built the complete pipeline using explicit LangChain components. The ten Wikipedia pages produced 3,573 chunks with the required 500/50 splitter, and all five questions had answer-bearing information inside the top three results.

The 100% retrieval success rate was therefore supported by manual passage inspection, while the Titanic and Matrix cases showed why first-relevant-rank analysis was still important. Changing to 300/75 chunking did not improve those two ranks.

For training optimization, the same controlled setup was used

to compare tensor placement, Kaiming initialization, activation checkpointing, gradient accumulation, and mixed precision. The results showed trade-offs rather than one configuration improving every metric.

The most useful lesson for me was that I needed to look at the actual output—not only whether code executed, whether a retrieved passage looked related, or whether a technique was expected to be an optimization. That verification process was what made the final findings understandable and defensible.

REFERENCES

- [1] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [2] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, “Learning word vectors for sentiment analysis,” in *Proc. 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 2011, pp. 142–150.
- [3] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [5] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.
- [6] T. Chen, B. Xu, C. Zhang, and C. Guestrin, “Training deep nets with sublinear memory cost,” *arXiv preprint arXiv:1604.06174*, 2016.
- [7] P. Micikevicius et al., “Mixed precision training,” in *Proc. International Conference on Learning Representations*, 2018.